ISA v2.1 — SM-Level WMMA + 4R4W Register File + Weight-Stationary PE

32 opcodes (0x00=NOP) | 4R4W LUT-based RF | 1-port memory (burst load/store) | 4×4 PE | WMMA.MMA rD,rA,rB,rC = 4 operands

1. Register File Specification — 4 Read + 4 Write Ports (LUT-based Distributed RAM)

Per-Thread Register File

Size: 16 entries × 16 bits = 256 bits
Read ports: 4 (combinational, via LUT muxes)
Write ports: 4 (clocked, synchronous — all 4 regs of a fragment row written in 1 cycle)
Technology: Distributed RAM (Xilinx LUT4)

Cost: 16 LUT4 per bit × 16 bits = 256 LUTs for 4R
(with sharing: ~64 LUTs effective per thread)

Total SM: 4 threads × ~64 LUTs = ~256 LUTs

Total bits: 4 × 256 = 1024 bits (all threads)

TID = special reg (hw per-lane, 0-3). R0-R15 = general purpose.

Why 4R4W

4 Read ports — single-cycle register access for all instructions:

R-type (ADD, MUL, etc.): 2 reads (rA, rB) — 2 of 4 ports used
U-type FMA: 3 reads (rA, rB, rD) — 3 of 4 ports used
WMMA.LOAD/STORE: uses MEMORY port (1 port, burst 4 cycles) — RF ports for addr only
WMMA.MMA gather: 4 reads per thread (full row) — all 4 RF ports used! RF→tensor core

4 Write ports — 1-cycle writeback from tensor core:
All 4 threads write {rD,rD+1,rD+2,rD+3} simultaneously in 1 clock. 4×4 = 16 regs written.

LUT-based distributed RAM is purely combinational for reads —
adding more read ports costs only extra mux logic, no extra BRAM.

2. SM-Level Collective Operation — WMMA.MMA Gathers from All 4 Thread Register Files

Following NVIDIA PTX model (adapted for 4-thread, 4×4 matrix):

1. WMMA.LOAD.A/B/C — each thread loads its own ROW fragment from memory into GPR {rD, rD+1, rD+2, rD+3}
2. WMMA.MMA — SM collective: gathers A+C from thread RFs → 4×4 PE → writes D back. C and D can be different reg groups.
3. WMMA.STORE — each thread stores its D row from GPR back to memory

SIMT lockstep guarantees all 4 threads at same instruction. No sync barrier needed (unlike NVIDIA .sync for divergent warps).

3. Fragment Distribution — Each Thread Owns One Row of Each Matrix

A[4×4] → R4-R7

T0: R4=a00 R5=a01 R6=a02 R7=a03 (row 0)
T1: R4=a10 R5=a11 R6=a12 R7=a13 (row 1)
T2: R4=a20 R5=a21 R6=a22 R7=a23 (row 2)
T3: R4=a30 R5=a31 R6=a32 R7=a33 (row 3)

Memory: row-major, each row = 8 bytes contiguous
Row stride = tid × 8 bytes

B[4×4] → R8-R11 (WEIGHT)

T0: R8=b00 R9=b01 R10=b02 R11=b03 (row 0)
T1: R8=b10 R9=b11 R10=b12 R11=b13 (row 1)
T2: R8=b20 R9=b21 R10=b22 R11=b23 (row 2)
T3: R8=b30 R9=b31 R10=b32 R11=b33 (row 3)

B stays valid as long as R8-R11 are not overwritten
Don't reload rB registers between WMMA.MMA calls to reuse weights

C/D[4×4] → R12-R15 (accumulator)

T0: R12=c/d00 R13=c/d01 R14=c/d02 R15=c/d03
T1: R12=c/d10 R13=c/d11 R14=c/d12 R15=c/d13
T2: R12=c/d20 R13=c/d21 R14=c/d22 R15=c/d23
T3: R12=c/d30 R13=c/d31 R14=c/d32 R15=c/d33

C: accumulator input (rC field). D: result output (rD field).
C and D can be same group (rC=rD) or different for flexibility.
All 16 GPRs used: R0(free) + R1-R3(addr) + R4-R15(frags). TID via MOV Rd, TID.

4. Weight-Stationary 4×4 PE — Internal Architecture + WMMA.MMA 37-Cycle Timing

PE Internal Storage (dedicated FFs, NOT in GPR)

B_weight[4][4] = 16 × 16-bit = 256 bits (gathered from rB regs during MMA)
A_col[4] = 4 × 16-bit = 64 bits (current k-step column)
D_accum[4][4] = 16 × 16-bit = 256 bits (running accumulator)

Total PE internal: 576 bits = 36 × 16-bit FFs

B_weight lifetime: Gathered from rB regs during WMMA.MMA.
Latched inside PE during compute.
WMMA.LOAD is just memory→register (no PE interaction).

WMMA.MMA Pipeline — Tensor Core = 37 Cycles (verified by testbench, 18/18 pass)

Gather (RF→TC)	Tensor Core: D = A×B + C — 37 cycles (pure compute)	WB (TC→RF)
Read rA, rB, rC from all 4 thread RFs via 4R ports → flat 256-bit vectors to tensor core	Inputs: matrix_a[255:0], matrix_b[255:0], matrix_c[255:0] (flat bit-vectors) Output: matrix_d[255:0] after 37 clock cycles (valid_in → valid_out) Internal: BF16 multiply-accumulate, 4×4×4 = 64 MACs Verified: 18 tests — identity, diagonal, negatives, cancellation, dot product, 2×2 matmul, b2bin 1 cycle (4×4=16 regs)	4W ports per thread: all 4 threads write {rD..+3} back to RF

37 cycles = pure tensor core execution (does NOT include fetch, decode, gather, or writeback).
Gather: RF→tensor core via 4R ports. WB: tensor core→RF via 4W ports (1 cycle, all 4 threads × 4 regs simultaneously).
Weight reuse: don't overwrite rB regs → skip WMMA.LOAD.B (save 4-cyc mem burst). MMA re-reads rB from RF each time.

Instruction Cycle Costs (1-port memory, 4R4W RF):

WMMA.LOAD.A/B/C (burst read, 1 mem port): 4 cycles each. Purely memory → register. All three identical hardware.
WMMA.STORE (burst write, 1 mem port): 4 cycles. Purely register → memory.
WMMA.MMA: gather(4R) + 37 cyc tensor core + 1 cyc WB(4W) = 37+ cycles for 64 MACs. 37 = pure compute (verified).

WMMA.LOAD.B cost amortized over many MMA calls in NN inference

5. WMMA Instruction Encodings — 3 New Opcodes

WMMA.LOAD — Burst Load Fragment Row (opcode 0x1E) — M-type — 4 cycles (1 memory port)

[31:27] opcode [26] DT=x [25:24] SEL [23:20] rD [19:16] rA [15:0] offset (DT don't-care: memory transfer is type-agnostic)

SEL=00: WMMA.LOAD.A Burst read 4 cycles: rD←MEM[rA+off], rD+1←MEM[+2], rD+2←MEM[+4], rD+3←MEM[+6]
SEL=01: WMMA.LOAD.B Same 4-cycle burst. SEL is assembler hint only — all three are identical hardware.
SEL=10: WMMA.LOAD.C Same 4-cycle burst → C/D accumulator init

Memory has 1 read port → burst reads 4 consecutive 16-bit words over 4 cycles.
WMMA.LOAD is purely memory → register. No tensor core / PE interaction.

WMMA.MMA — SM Collective (opcode 0x1D) — R-type extended — 4 register group operands

{rD..+3} = matmul({rA..+3}, {rB..+3}) + {rC..+3}

[23:20] rD=output [19:16] rA=input [15:12] rB=weight [11:8] rC=accumulator. All independent.

4R ports per thread RF → gathers from all thread RFs. 37 cycles total. No memory access.

WMMA.STORE — Burst Store Result Fragment Row (opcode 0x1F) — M-type — 4 cycles (1 memory port)

MEM[rA+off]←rD, MEM[+2]←rD+1, MEM[+4]←rD+2, MEM[+6]←rD+3

Burst store 4 × 16-bit over 4 cycles (1 mem write port). Thread n stores its D row n.

6. Complete Opcode Table — 32 Active Opcodes

HEX	BIN	MNEMONIC	TYPE	SEMANTICS	DT	RF PORTS USED	DECODER NOTE
0x00	00000	**NOP**	—	no operation (safe default for zeroed memory)	—	0	
0x01	00001	**ST**	M	MEM[rA + offset] ← rD	—	2R(rA,rD)	⚠ rD=source
0x02	00010	**LD**	M	rD ← MEM[rA + offset]	—	1R(rA) + 1W(rD)	
0x03	00011	**MOV**	U	rD ← rA (bit24=0) | rD ← TID (bit24=1)	—	1R + 1W	
0x04	00100	**MOVI**	P	rD ← imm16	—	1W	
0x05	00101	**CVT**	U	rD ← convert(rA)	0/1	1R + 1W	
0x06	00110	**ADD**	R	rD ← rA + rB	0/1	2R + 1W	
0x07	00111	**SUB**	R	rD ← rA - rB	0/1	2R + 1W	
0x08	01000	**MUL**	R	rD ← rA × rB	0/1	2R + 1W	
0x09	01001	**FMA**	U	rD ← rA × rB + rD	0/1	3R(rA,rB,rD) + 1W	⚠ rD r+w
0x0A	01010	**MAX**	R	rD ← max(rA, rB)	0/1	2R + 1W	
0x0B	01011	**MIN**	R	rD ← min(rA, rB)	0/1	2R + 1W	
0x0C	01100	**ABS**	U	rD ← |rA|	0/1	1R + 1W	
0x0D	01101	**NEG**	U	rD ← -rA	0/1	1R + 1W	
0x0E	01110	**AND**	R	rD ← rA & rB	0	2R + 1W	
0x0F	01111	**OR**	R	rD ← rA | rB	0	2R + 1W	
0x10	10000	**XOR**	R	rD ← rA ^ rB	0	2R + 1W	
0x11	10001	**SHL**	I	rD ← rA << imm	0	1R + 1W	
0x12	10010	**SHR**	I	rD ← rA >> imm	0	1R + 1W	
0x13	10011	**ADDI**	I	rD ← rA + imm	0/1	1R + 1W	
0x14	10100	**MULI**	I	rD ← rA × imm	0/1	1R + 1W	
0x15	10101	**SETP**	C	P[n] ← (rA cmp rB)	0/1	2R	⚠ pred RF
0x16	10110	**SELP**	R	rD ← P[n] ? rA : rB	0/1	2R + 1W	
0x17	10111	**BRA**	P	PC ← target	—	0	
0x18	11000	**PBRA**	P	if P[n]: PC ← target	—	0	
0x19	11001	**RET**	U	end kernel	—	0	
0x1A	11010	**SET**	C	rD ← (rA cmp rB) ? 1 : 0	0/1	2R + 1W	
0x1B	11011	**LDS**	M	rD ← SMEM[rA + offset]	—	1R + 1W	
0x1C	11100	**STS**	M	SMEM[rA + offset] ← rD	—	2R	⚠ rD=source
0x1D	11101	**WMMA.MMA**	R	{rD..+3}=matmul({rA..+3},{rB..+3})+{rC..+3}	1	4R(rA,rB,rC)×4T + 4W(rD)×4T	★ SM 4-op 37cyc
0x1E	11110	**WMMA.LOAD**	M	{rD..+3}←MEM[rA+off] SEL:A/B/C	—	1R(rA) + 4W burst (4cyc)	★ burst mem→reg
0x1F	11111	**WMMA.STORE**	M	MEM[rA+off]←{rD..+3}	—	4R(rD..+3) burst (4cyc)	★ burst store

32 active opcodes / 32 possible. 0x00=NOP (safe default for zeroed memory). WMMA.MMA is multi-cycle (37 cyc for 64 MACs).

7. Decoder Exceptions (5 total)

⚠ #1: ST / STS (0x01, 0x1C) — rD field is READ source (store data), not write destination
⚠ #2: FMA (0x09) — rD is BOTH read (accumulator) and written. Uses 3 of 4 read ports.
⚠ #3: SETP (0x15) — rD selects predicate register P[n], not GPR. Write goes to predicate RF.
⚠ **#4: WMMA.MMA (0x1D) — SM collective, 37-cycle. 4 reg group operands: rD(out), rA(input), rB(weight), rC(accum). All from RF.**
⚠ **#5: WMMA.LOAD/STORE (0x1E/0x1F) — 4-cycle burst (1 mem port): rD indexes {rD,rD+1,rD+2,rD+3}. Purely memory ↔ register.**

8. Complete Kernel 6: 4×4×4 bf16 WMMA Matmul — 16 Instructions

LN	OUR ISA	BINARY [31:0]	HEX	EFFECT
	── SETUP ──			
1	MOVI R1, base_A	00100 0 00 0001 0000 xxxxxxxxxxxxxxxx	0x2010_xxxx	P-type DT=0. Global A base
2	MOVI R2, base_B	00100 0 00 0010 0000 xxxxxxxxxxxxxxxx	0x2020_xxxx	Global B base
3	MOVI R3, base_D	00100 0 00 0011 0000 xxxxxxxxxxxxxxxx	0x2030_xxxx	Global D base
4	MOV R4, TID	00011 0 01 0100 0000 0000000000000000	0x1940_0000	U-type bit24=1. R4 = lane id (0-3)
5	SHL R4, R4, 3	10001 0 00 0100 0100 0000000000000011	0x8844_0003	I-type. R4 = tid × 8 (row stride = 4×2B)
7	ADD R2, R2, R4	00110 0 00 0010 0010 0100 000000000000	0x3022_4000	R2 = &B[my_row][0]
8	ADD R3, R3, R4	00110 0 00 0011 0011 0100 000000000000	0x3033_4000	R3 = &D[my_row][0]
	── WMMA LOAD FRAGMENTS ──			
9	WMMA.LOAD.A R4, R1, 0	11110 1 00 0100 0001 0000000000000000	0xF041_0000	SEL=00. {R4-R7}←A[my_row] 4-cyc burst (1 mem port)
10	WMMA.LOAD.B R8, R2, 0	11110 1 01 1000 0010 0000000000000000	0xF482_0000	SEL=01. {R8-R11}←B[my_row] 4-cyc burst
	── ZERO C ACCUMULATORS ──			
11	MOVI R12, 0x0000	00100 1 00 1100 0000 0000000000000000	0x24C0_0000	P-type DT=1. C[my_row][0]=0
12	MOVI R13, 0x0000	00100 1 00 1101 0000 0000000000000000	0x24D0_0000	C[my_row][1]=0
13	MOVI R14, 0x0000	00100 1 00 1110 0000 0000000000000000	0x24E0_0000	C[my_row][2]=0
14	MOVI R15, 0x0000	00100 1 00 1111 0000 0000000000000000	0x24F0_0000	C[my_row][3]=0
	── WMMA COMPUTE (SM COLLECTIVE — 37 CYCLES, 64 MACs) ──			
15	WMMA.MMA R12, R4, R8, R12	11101 1 00 1100 0100 1000 1100 00000000	0xECC4_8C00	★ {R12-15}=A×B+C 37cyc 64 MACs!
	── WMMA STORE + RET ──			
16	WMMA.STORE R12, R3, 0	11111 1 00 1100 0011 0000000000000000	0xFCC3_0000	{R12-R15}→MEM[R3] 4-cyc burst (1 mem port)
17	RET	11001 0 00 0000 0000 0000000000000000	0xC800_0000	U-type. End kernel.

K6: 17 instructions | 8 setup (MOV TID + 3 MOVI + SHL + 3 ADD) + 2 WMMA.LOAD(4cyc burst each) + 4 zero-C + 1 WMMA.MMA(37cyc TC + gather/WB) + 1 WMMA.STORE(4cyc burst) + 1 RET
WMMA.LOAD/STORE: 1 memory port → 4-cycle burst. WMMA.MMA: 37 cyc pure tensor core (verified 18/18 testbench). 4W ports → 1-cycle WB.